Project 1: Forecasting South African Accommodation Demand with ARIMA and SARIMA Models

Section 1: ARIMA and seasonal ARIMA models

Eden

Table of contents

Project summary 1

1 Introduction and research question 2

2 Data 3
2.1 Source and data statement . . . 3
2.2 Reading and reshaping the data . . . 3

3 Exploratory data analysis 5
3.1 Time plot . . . 5
3.2 Seasonal patterns . . . 6
3.3 STL decomposition . . . 7
3.4 Hotels compared with the total industry . . . 8
3.5 Stationarity and identification (pre-COVID data) . . . 9

4 Modelling strategy 11
4.1 The Box and Jenkins workflow . . . 11
4.2 The COVID problem . . . 12
4.3 Training and test sets . . . 12
4.4 Candidate models . . . 13

5 Model fitting 13
5.1 Selected orders and information criteria . . . 14
5.2 Residual diagnostics . . . 16

6 Forecast evaluation on the test set 20

7 Rolling-origin cross-validation 23

8 Final model and forecasts 25

9 Discussion 27
9.1 Key findings . . . 27

9.2 Issues and limitations 27
9.3 What I would do differently 28

Reproducibility **28**

Project summary

Executive summary

Title: Forecasting South African accommodation demand with ARIMA and SARIMA models
Section and topic: Section 1. Benchmarks, stationarity and differencing, ARMA, ARIMA and SARIMA models, the Box and Jenkins workflow, forecast evaluation.
Type of project: Implementation and model comparison on real-world data.
Question: How much forecast accuracy is gained by moving from simple benchmark methods to ARMA, ARIMA and seasonal ARIMA models for monthly stay unit nights sold in South Africa, and how sensitive is that gain to the way the COVID-19 structural break is handled?
Data: Real-world, publicly available data from Statistics South Africa (Stats SA), *Tourist Accommodation* statistical release P6410, series B64100 (stay unit nights sold, total industry, thousands), monthly. Analysis window January 2015 to June 2026. Full source details are in Section 2.1.
Methods: Four benchmarks (mean, naive, seasonal naive, drift); ARMA, ARIMA and SARIMA models on the log scale, including an automatically selected SARIMA and a SARIMA identified by hand from the ACF and PACF. Every model is fitted under two designs: (A) the COVID months are kept as observed, and (B) the COVID months (March 2020 to March 2022) are treated as missing values and handled by the Kalman filter inside the ARIMA likelihood. Models are compared on a 24 month hold-out test set (RMSE, MAE, MAPE, MASE, interval coverage) and by expanding window rolling-origin cross-validation.
Key findings: See Section 9.1. In short: with the COVID months left in the data, the seasonal ARIMA models are no better than the seasonal naive benchmark, because the collapse in April 2020 dominates the estimated variance and distorts the parameters. Treating the COVID window as missing gives a stable SARIMA$(0,1,1)(0,1,1)_{12}$ "airline" model with well behaved residuals, and that model gives the most accurate forecasts on both the test set and in cross-validation.
What I would do differently: Model the shock explicitly with intervention variables (planned for Project 3), and compare the expanding window with a sliding window that forgets the pre-COVID years.
Detailed analysis: the sections below.

1 Introduction and research question

Tourist accommodation is one of the clearest seasonal series published by Statistics South Africa. Demand peaks over the December holidays and dips in the winter months, and in April 2020 it collapsed almost to zero when the hard lockdown closed the industry. This makes it a good series for the Box and Jenkins material: the seasonality calls for seasonal differencing and a SARIMA model, while the COVID shock is a real example of a structural break that violates the assumptions those models are built on.

Research question. *How much forecast accuracy is gained by moving from benchmark methods to ARMA, ARIMA and SARIMA models for monthly stay unit nights sold, and how sensitive is this gain to the treatment of the COVID-19 break?*

The project follows the five stages of a forecasting project from the notes: define the problem, gather the data, explore it, choose and fit models, and evaluate the forecasts on data the models have not seen.

2 Data

2.1 Source and data statement

Data source at a glance

Publisher	Statistics South Africa (Stats SA)
Publication	*Tourist Accommodation*, statistical release P6410 (monthly)
Where to get it	Stats SA website, P6410 time series data: https://www.statssa.gov.za/?page_id=1854&PPN=P6410
File used	`tourist_data.xlsx` (Stats SA time series spreadsheet for P6410)
Series analysed	B64100: Stay units nights sold, Total industry, actual figures, thousands
Comparison series	B64101: Stay units nights sold, Hotels
Coverage of file	January 2007 to June 2026
Date downloaded	*[add the date you downloaded the file]*
Licence and privacy	Public, aggregated official statistics; no personal information, so POPIA does not apply

Reference: Statistics South Africa (2026) *Tourist Accommodation (Statistical release P6410)*, time series data, January 2007 to June 2026. Pretoria: Statistics South Africa. Available at: https://www.statssa.gov.za/?page_id=1854&PPN=P6410 (Accessed: *[date]*).

The data come from the Statistics South Africa Tourist Accommodation release (P6410), downloaded as the time series spreadsheet from the Stats SA website. The file holds monthly series from January 2007 to June 2026 for stay units available, stay unit nights sold, occupancy rates and income, by type of accommodation. The data are aggregated industry totals, so there is no personal information and POPIA does not apply. The variable analysed is **B64100: stay unit nights sold, total industry, actual (not seasonally adjusted) figures, in thousands**. The hotels series (B64101) is used once in the exploratory analysis for comparison.

2.2 Reading and reshaping the data

The Stats SA file has one row per series and one column per month, with month columns named `MOmmyyyy`. The code below reshapes it to long format and builds a `tsibble`.

```r
raw <- read_excel(data_file)

tourism_long <- raw |>
  filter(H03 %in% c("B64100", "B64101")) |>
  select(H03, H04, starts_with("M0")) |>
  pivot_longer(starts_with("M0"), names_to = "code", values_to = "nights") |>
  mutate(
    Month    = yearmonth(make_date(as.integer(substr(code, 5, 8)),
                                   as.integer(substr(code, 3, 4)), 1L)),
    nights   = as.numeric(nights),
    industry = if_else(H03 == "B64100", "Total industry", "Hotels")
  ) |>
  filter(!is.na(nights)) |>
  select(Month, industry, nights)

# Total industry, January 2015 onwards, with a COVID period label
nights_sold <- tourism_long |>
  filter(industry == "Total industry") |>
  select(Month, nights) |>
  as_tsibble(index = Month) |>
  filter_index("2015 Jan" ~ .) |>
  mutate(period = case_when(
    Month <  covid_start ~ "Pre-COVID",
    Month <= covid_end   ~ "COVID window",
    TRUE                 ~ "Post-COVID"
  ))

pre_covid <- nights_sold |> filter(Month < covid_start)

has_gaps(nights_sold)
```

```
# A tibble: 1 x 1
  .gaps
  <lgl>
1 FALSE
```

```r
nights_sold |>
  as_tibble() |>
  mutate(period = factor(period, levels = c("Pre-COVID", "COVID window",
   ↪  "Post-COVID"))) |>
  group_by(period) |>
  summarise(
    From    = format(min(Month)),
    To      = format(max(Month)),
    Months  = n(),
    Mean    = mean(nights),
    Minimum = min(nights),
    Maximum = max(nights)
  ) |>
  kable(digits = 0, format.args = list(big.mark = ","))
```

Table 2: Overview of the analysis series (thousands of stay unit nights sold).

period	From	To	Months	Mean	Minimum	Maximum
Pre-COVID	2015 Jan	2020 Feb	62	2,268	1,933	2,581
COVID window	2020 Mar	2022 Mar	25	981	57	1,627
Post-COVID	2022 Apr	2026 Jun	51	1,813	1,466	2,133

The series has no implicit gaps. The analysis starts in January 2015 so that the pre-COVID part of the series (62 months) reflects the recent structure of the industry rather than the period straight after the 2008 financial crisis and the 2010 World Cup.

3 Exploratory data analysis

3.1 Time plot

```
full_df <- tourism_long |>
  filter(industry == "Total industry") |>
  mutate(date = as.Date(Month))

ggplot(full_df, aes(date, nights)) +
  annotate("rect",
           xmin = as.Date(covid_start), xmax = as.Date(covid_end),
           ymin = -Inf, ymax = Inf, alpha = 0.15, fill = "firebrick") +
  geom_vline(xintercept = as.Date("2015-01-01"), linetype = "dashed", colour =
   ↪  "grey40") +
  geom_line(colour = "steelblue4") +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = NULL, y = "Nights sold (thousands)")
```

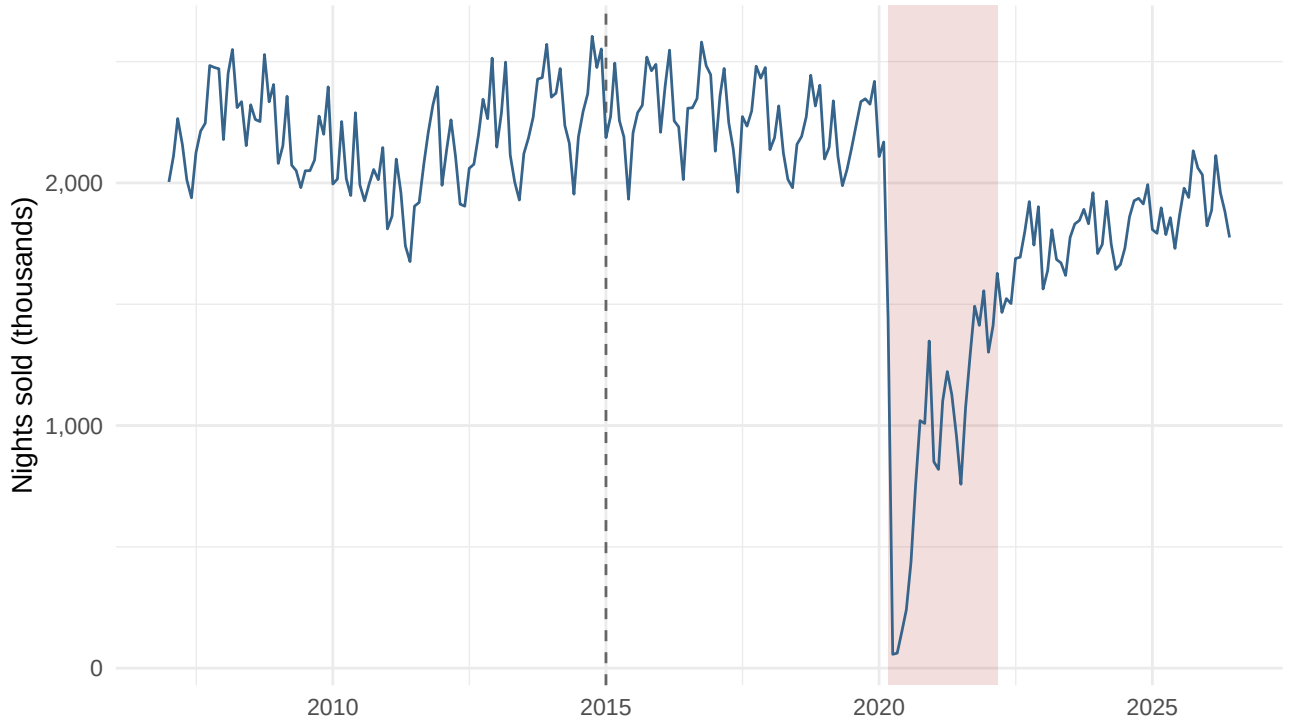


Figure 1: Monthly stay unit nights sold, total industry, January 2007 to June 2026. The shaded band is the COVID window (March 2020 to March 2022); the dashed line marks the start of the analysis window.

Figure 1 shows three distinct regimes. Before 2020 the series moves around a fairly flat level of about 2,212 thousand nights a month (2019 average) with a strong and regular seasonal pattern. In April 2020 nights sold fell to 57 thousand, 2.7% of April 2019, and the recovery took roughly two years, with setbacks during the later waves. Over the most recent 12 months the average is 1,954 thousand, close to but still below the pre-COVID level. The series is clearly not stationary in mean over the full window, and the variability is much larger during the COVID window than anywhere else.

3.2 Seasonal patterns

```
nights_sold |>
  filter(period != "COVID window") |>
  fill_gaps() |>
  gg_season(nights, labels = "right") +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(y = "Nights sold (thousands)", x = NULL)
```

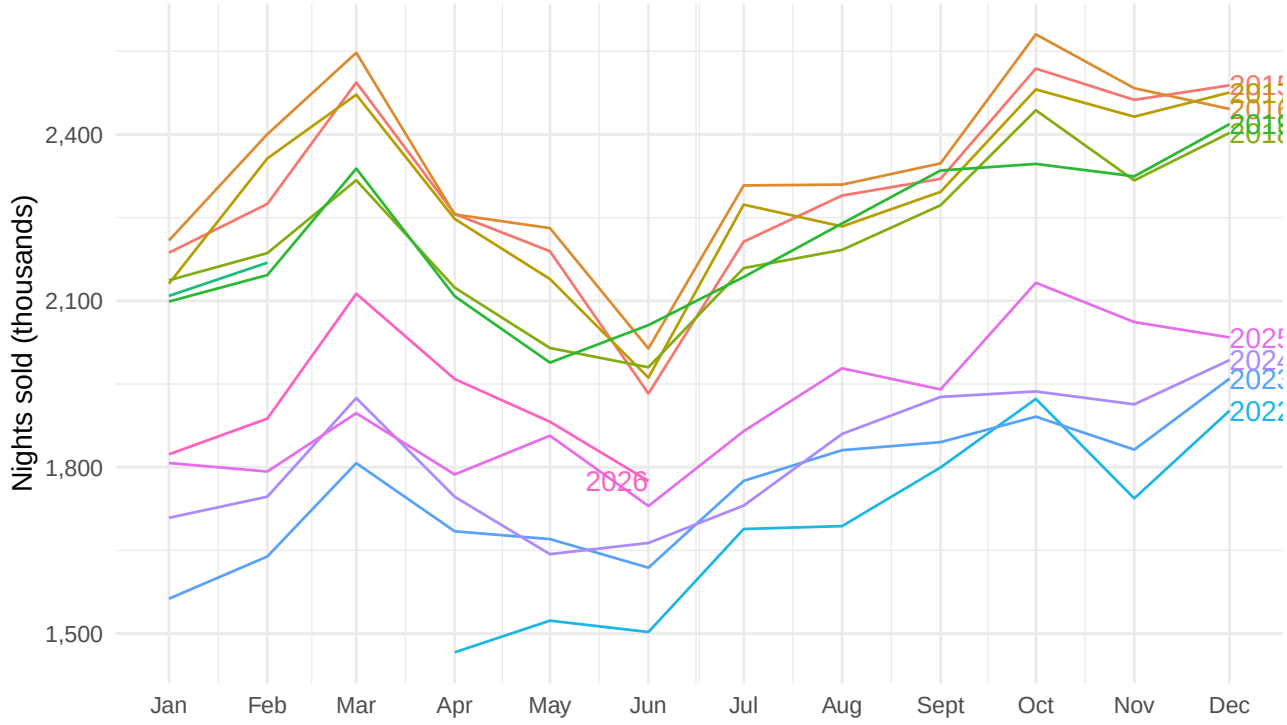


Figure 2: Seasonal plot of nights sold by year (pre-COVID and post-COVID years only).

```
pre_covid |>
  gg_subseries(nights) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(y = "Nights sold (thousands)", x = NULL)
```

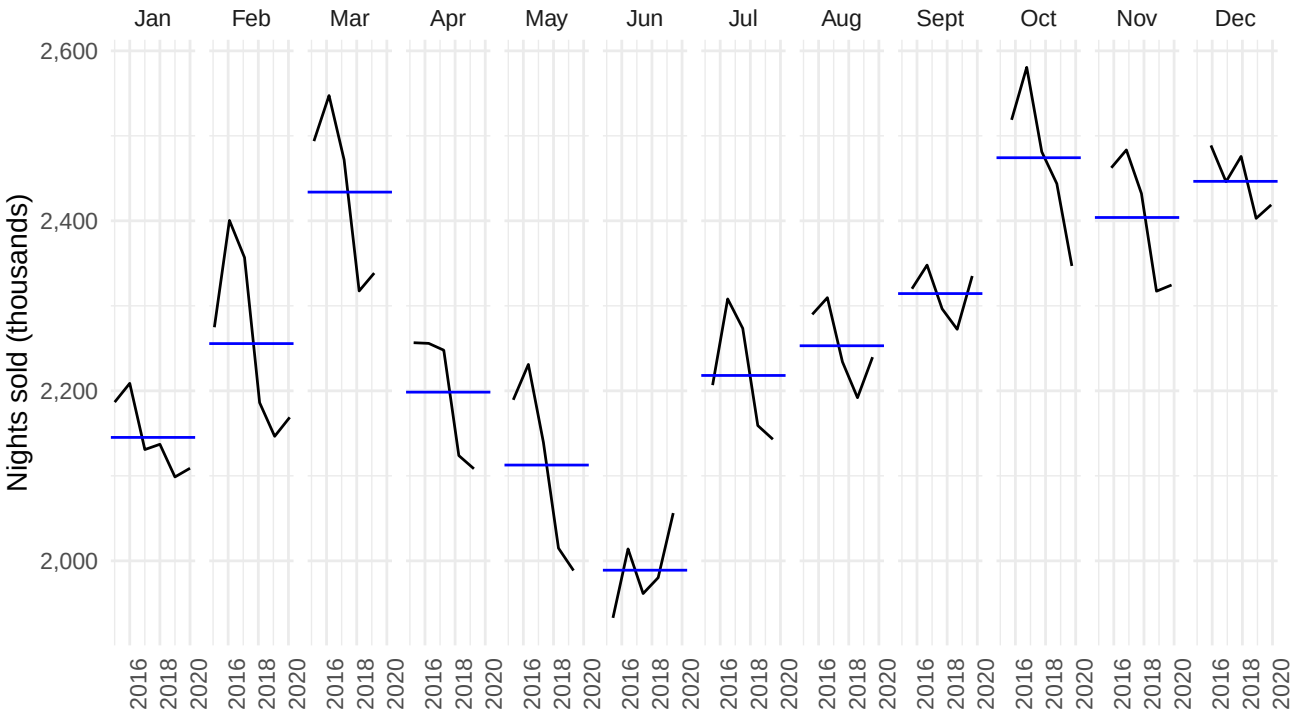


Figure 3: Seasonal subseries plot for the pre-COVID years (2015 to 2019). Blue lines are the monthly means.

Figure 2 and Figure 3 show that the *shape* of the seasonal pattern is very stable from year to year: a dip in January, a local peak in March (Easter and school holidays), a low in May and June, then a steady rise to the highest values in October to December. The post-COVID years sit lower but follow the same shape. This matters for modelling: a stable seasonal shape suggests a seasonal difference (or a seasonal MA term close to the boundary), and it means that the seasonal naive method is a strong benchmark.

3.3 STL decomposition

```
nights_sold |>
  model(STL(log(nights) ~ trend(window = 13) + season(window = "periodic"),
            robust = TRUE)) |>
  components() |>
  autoplot() +
  labs(x = NULL)
```

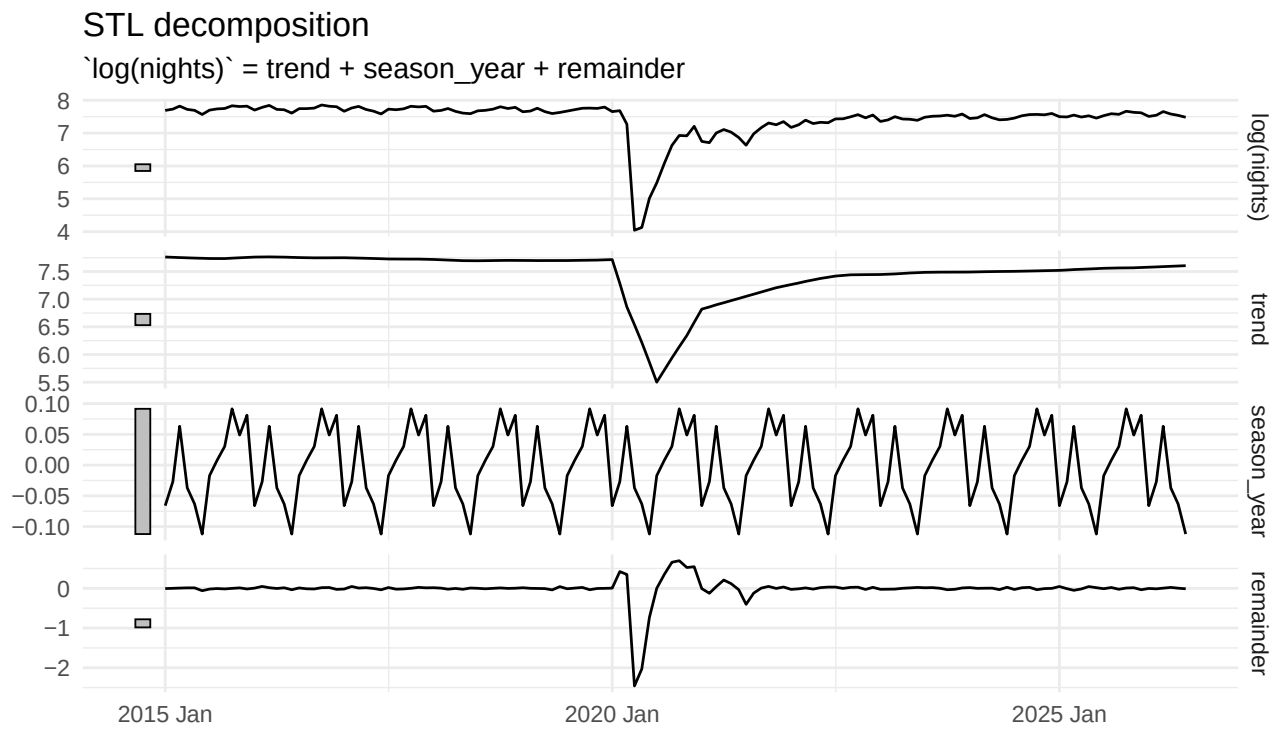


Figure 4: Robust STL decomposition of log nights sold, January 2015 to June 2026, with a fixed (periodic) seasonal component.

The log scale is used because the seasonal swings are roughly proportional to the level (they shrank when the level fell in 2020 and 2021), so a log transformation turns multiplicative seasonality into additive seasonality. On the pre-COVID data the strength of seasonality is 0.93 and the strength of trend is 0.64 (both measured from 0 to 1), so seasonality is the dominant feature. In Figure 4 the COVID collapse is absorbed almost entirely by the trend and the remainder, while the seasonal component is unaffected.

3.4 Hotels compared with the total industry

```
tourism_long |>
  pivot_wider(names_from = industry, values_from = nights) |>
  filter(Month >= yearmonth("2015 Jan")) |>
  mutate(period = case_when(
    Month <  covid_start ~ "Pre-COVID",
    Month <= covid_end   ~ "COVID window",
    TRUE                 ~ "Post-COVID"
  )) |>
  ggplot(aes(`Total industry`, Hotels, colour = period)) +
  geom_point(alpha = 0.8) +
  scale_colour_manual(values = c("Pre-COVID" = "steelblue4",
                                 "COVID window" = "firebrick",
                                 "Post-COVID" = "darkorange")) +
  scale_x_continuous(labels = scales::label_comma()) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = "Total industry (thousands)", y = "Hotels (thousands)", colour = NULL)
```

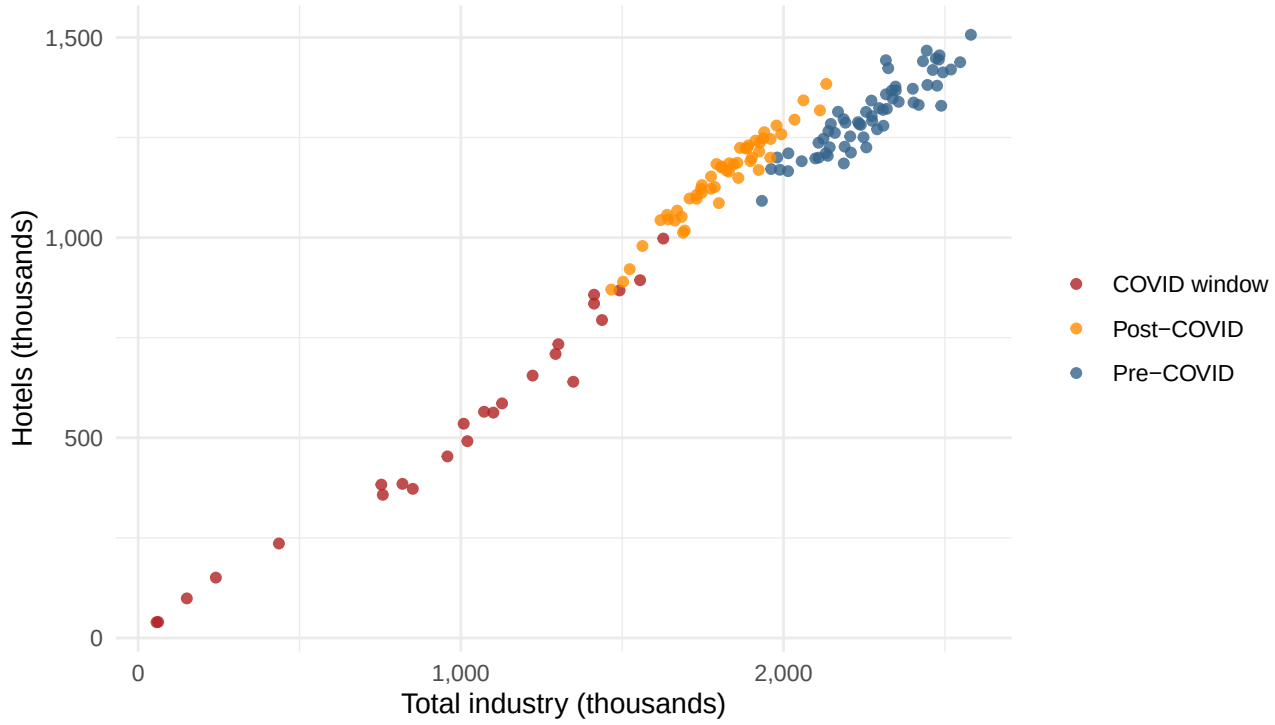


Figure 5: Hotel nights sold against total industry nights sold, coloured by period.

Hotels account for more than half of all nights sold and move almost one for one with the total, in every period (Figure 5). The COVID points simply slide down the same line rather than forming a separate cloud, so the shock hit all accommodation types together. This supports modelling the total industry series on its own.

3.5 Stationarity and identification (pre-COVID data)

Identification is done on the pre-COVID data only, because the COVID collapse would dominate any ACF calculated over the full series and hide the ordinary dependence structure.

```
lx  <- log(pre_covid$nights)
d12 <- diff(lx, lag = 12)

kpss_lx  <- unitroot_kpss(lx)
kpss_d12 <- unitroot_kpss(d12)
nsd_lx   <- unname(unitroot_nsdiffs(lx, .period = 12))
nd_lx    <- unname(unitroot_ndiffs(lx))
nd_d12   <- unname(unitroot_ndiffs(d12))

tibble(
  Series           = c("log nights", "seasonally differenced log nights"),
  `KPSS statistic` = c(kpss_lx["kpss_stat"], kpss_d12["kpss_stat"]),
  `KPSS p-value`   = c(kpss_lx["kpss_pvalue"], kpss_d12["kpss_pvalue"]),
  `First differences suggested (ndiffs)` = c(nd_lx, nd_d12),
  `Seasonal differences suggested (nsdiffs)` = c(nsd_lx, NA)
```

```
) |>
  kable(digits = 3)
```

Table 3: Unit root tests on log nights sold, January 2015 to February 2020.

Series	KPSS statistic	KPSS p-value	First differences suggested (ndiffs)	Seasonal differences suggested (nsdiffs)
log nights	0.248	0.1	0	1
seasonally differenced log nights	0.326	0.1	0	NA

The seasonal strength test suggests 1 seasonal difference for the log series. After the seasonal difference, Figure 6 still shows a slowly decaying ACF over the first several lags, the classic sign that the level is wandering, and the KPSS based `ndiffs` on the seasonally differenced series suggests 0 further first difference. The KPSS p-values are interpolated from a table that only runs from 0.01 to 0.1, so values at those limits should be read as "at most" or "at least". With only 62 pre-COVID months these tests have limited power, so the decision is based on the plots as well as the tests.

```
pre_covid |>
  gg_tsdisplay(difference(log(nights), 12), plot_type = "partial", lag_max = 36)
```

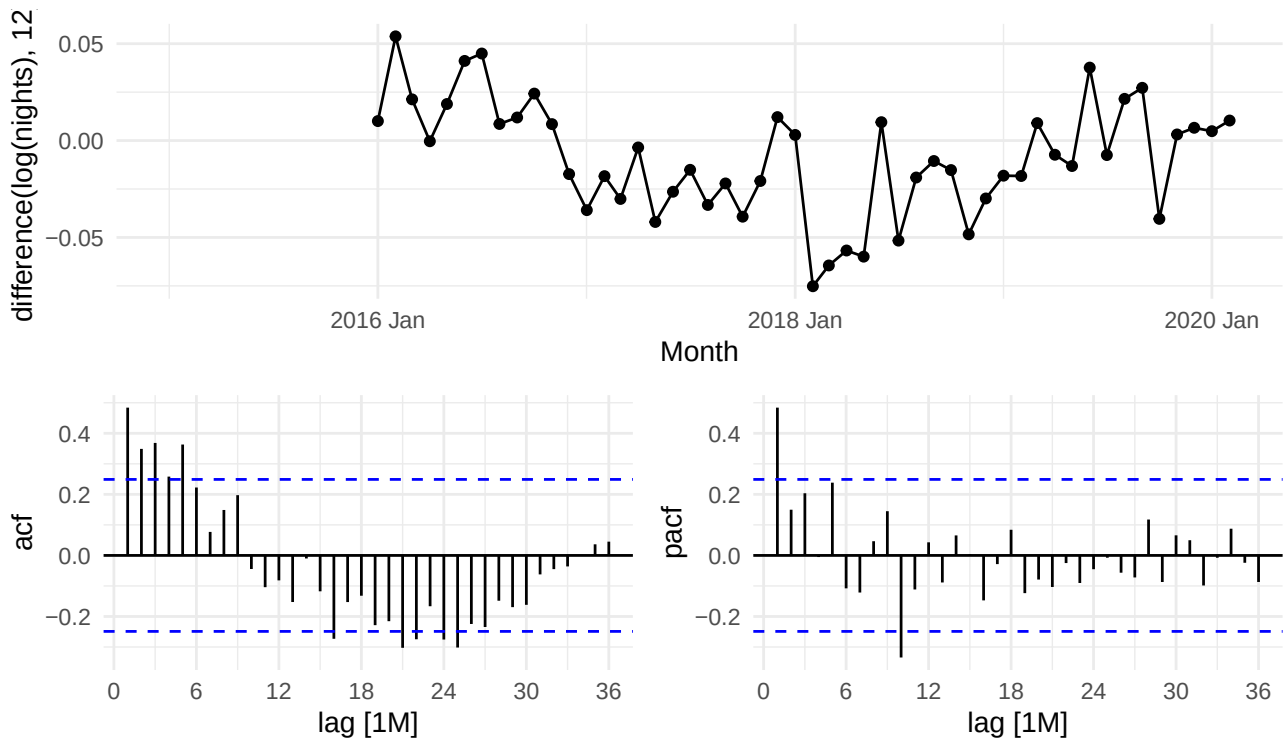


Figure 6: Seasonally differenced log nights sold (pre-COVID) with its ACF and PACF.

```
pre_covid |>
  gg_tsdisplay(difference(difference(log(nights), 12)), plot_type = "partial",
   ↪  lag_max = 36)
```

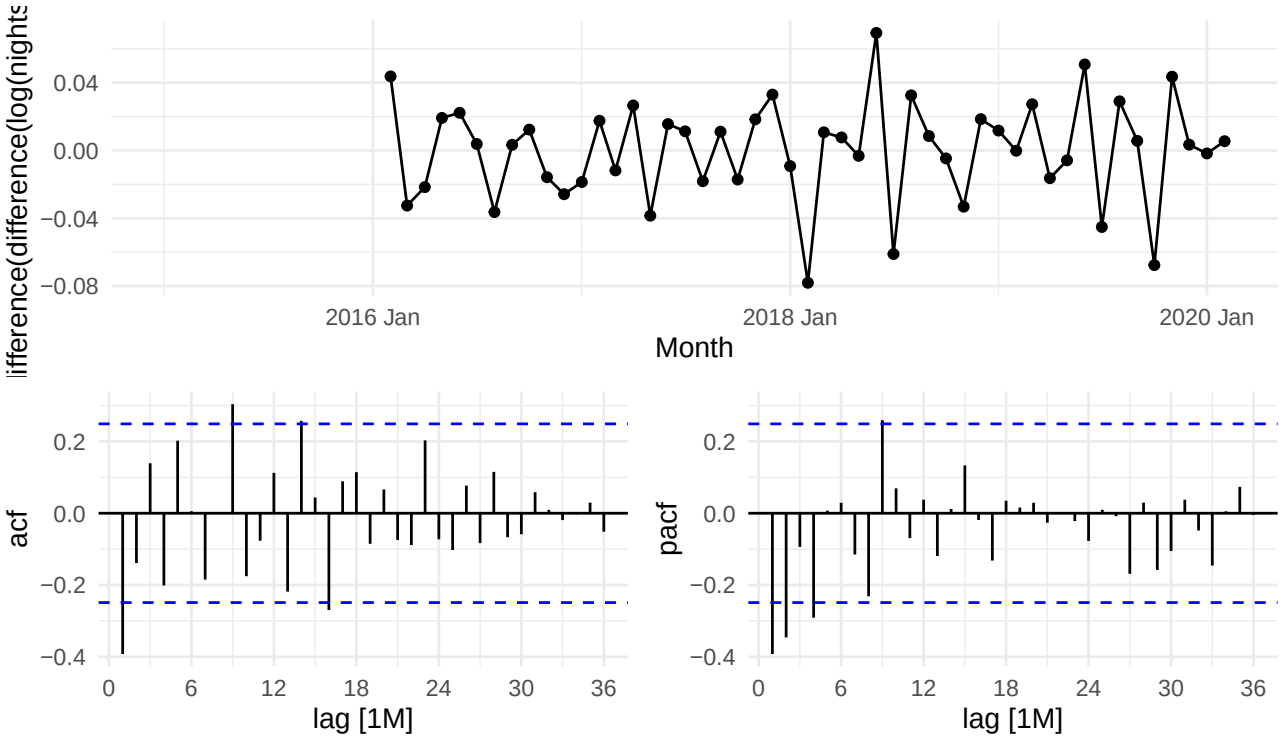


Figure 7: Seasonally and first differenced log nights sold (pre-COVID) with its ACF and PACF.

After both differences (Figure 7) the series looks stationary. The ACF has a clear negative spike at lag 1 (r_1 = -0.39) and a negative spike near lag 12 ($r_{12} = 0.11$), while the PACF tails off over the first few lags. Spikes at lag 1 and lag 12 in the ACF with a decaying PACF point to an MA(1) term and a seasonal MA(1) term. Following the Box and Jenkins workflow, the hand identified model is

$$(1-L)(1-L^{12})\log y_t = (1+\theta_1 L)(1+\Theta_1 L^{12})\,\varepsilon_t, \qquad \varepsilon_t \sim \mathrm{WN}(0,\sigma^2),$$

the SARIMA$(0,1,1)(0,1,1)_{12}$ or "airline" model. No constant is included because with d + D = 2 a constant would imply a quadratic trend in the forecasts.

4 Modelling strategy

4.1 The Box and Jenkins workflow

The modelling follows the Box and Jenkins workflow from the notes. Each step maps to a part of this report:

1. **Plot the data:** the time plot, seasonal plots and STL decomposition in the exploratory analysis.
2. **Difference until approximately stationary:** one seasonal and one first difference of log nights sold, chosen from the unit root tests and the ACF.
3. **Read the ACF and PACF to identify candidate orders:** the lag 1 and lag 12 spikes point to the SARIMA$(0,1,1)(0,1,1)_{12}$ model. The automatic search and the simpler ARMA and ARIMA models are included as alternative candidates.

4. **Estimate by maximum likelihood:** `ARIMA()` in `fable`, under both treatments of the COVID months.
5. **Diagnose the residuals against the white noise standard:** Ljung and Box tests, residual plots, ACF and histogram.
6. **Forecast only if the diagnostics pass; otherwise return to step 3.**

Finally, a model that survives the loop must still beat the benchmarks on the test set before it earns its place. In this report, models that fail step 5 are still forecast on the test set, but only as reference points showing *why* the loop matters. They are not candidates for the final model.

4.2 The COVID problem

ARIMA models assume that one set of parameters describes the whole series. The COVID window breaks this in two ways. First, the April 2020 collapse produces residuals many times larger than normal, which inflates $\hat{\sigma}^2$ and therefore every prediction interval. Second, the estimation tries to explain the shock and the recovery with the same MA and AR terms that describe normal month-to-month behaviour, which pulls the coefficients away from their usual values. Simply deleting the COVID months is not possible, because an ARIMA model needs equally spaced observations.

Two designs are compared:

- **Design A (COVID kept):** fit every model to the observed series, including the COVID months.
- **Design B (COVID as missing):** replace March 2020 to March 2022 by `NA`. The ARIMA likelihood is computed with the Kalman filter, which skips missing observations and simply runs the prediction step through the gap, so the parameters are estimated only from the pre-COVID and post-COVID months while the time ordering is kept intact. The benchmark methods also cope with missing values (for example, seasonal naive uses the last observed value for each month).

This is a deliberately simple treatment of the break. A more structural treatment with intervention variables is left for Project 3.

4.3 Training and test sets

```
train_A <- nights_sold |> filter(Month < test_start)
train_B <- train_A |>
  mutate(nights = if_else(period == "COVID window", NA_real_, nights))
test    <- nights_sold |> filter(Month >= test_start)

design_levels <- c("A: COVID kept", "B: COVID as missing")

train <- bind_rows(
  as_tibble(train_A) |> mutate(design = design_levels[1]),
  as_tibble(train_B) |> mutate(design = design_levels[2])
) |>
  as_tsibble(index = Month, key = design)

# Full observed series with the design key, used for accuracy() so that
# both designs are scored against the same actual values and the MASE
# uses the same scaling for both designs.
```

```
nights_keyed <- bind_rows(
  as_tibble(nights_sold) |> mutate(design = design_levels[1]),
  as_tibble(nights_sold) |> mutate(design = design_levels[2])
) |>
  as_tsibble(index = Month, key = design)
```

The models are trained on January 2015 to June 2024 (114 months, of which 25 are set to missing in Design B) and tested on July 2024 to June 2026 (24 months). A two year test set covers two full seasonal cycles, so every calendar month is forecast twice, at short and long horizons.

4.4 Candidate models

Table 4: Candidate models. The ARMA, ARIMA and SARIMA models form a ladder in which each step adds one feature of the data.

Model	Specification	Role
Mean	$\hat{y}_{T+h} = \bar{y}$	Benchmark
Naive	$\hat{y}_{T+h} = y_T$	Benchmark
Seasonal naive	$\hat{y}_{T+h} = y_{T+h-12k}$	Benchmark
Drift	Naive plus average historical change	Benchmark
ARMA	ARMA(p, q) with constant on $\log y_t$, orders by AICc, no differencing	Ignores trend and seasonality
ARIMA	ARIMA(p, 1, q) on $\log y_t$, orders by AICc	Handles the level, ignores seasonality
SARIMA (auto)	ARIMA(p, 1, q)(P, 1, Q)$_{12}$ on $\log y_t$, orders by AICc	Automatic seasonal model
SARIMA (0,1,1)(0,1,1)	Hand identified airline model on $\log y_t$	Box and Jenkins model

The differencing orders are fixed from the identification step rather than chosen by unit root tests inside `ARIMA()`, so that both designs use the same differencing and the comparison isolates the effect of the COVID treatment. Point forecasts from the log scale models are back-transformed with the bias adjustment that `fable` applies by default, so they are forecasts of the mean rather than the median.

5 Model fitting

```
fit <- train |>
  model(
    Mean        = MEAN(nights),
    Naive       = NAIVE(nights),
    SNaive      = SNAIVE(nights),
    Drift       = RW(nights ~ drift()),
    ARMA        = ARIMA(log(nights) ~ 1 + pdq(d = 0) + PDQ(0, 0, 0)),
    ARIMA       = ARIMA(log(nights) ~ pdq(d = 1) + PDQ(0, 0, 0)),
```

```
    SARIMA_auto = ARIMA(log(nights) ~ pdq(d = 1) + PDQ(D = 1)),
    SARIMA_BJ   = ARIMA(log(nights) ~ 0 + pdq(0, 1, 1) + PDQ(0, 1, 1))
  )

arima_models <- c("ARMA", "ARIMA", "SARIMA_auto", "SARIMA_BJ")

model_labels <- c(
  Mean = "Mean", Naive = "Naive", SNaive = "Seasonal naive", Drift = "Drift",
  ARMA = "ARMA", ARIMA = "ARIMA", SARIMA_auto = "SARIMA (auto)",
  SARIMA_BJ = "SARIMA (0,1,1)(0,1,1)"
)
```

5.1 Selected orders and information criteria

```
orders <- fit |>
  select(design, all_of(arima_models)) |>
  pivot_longer(-design, names_to = ".model", values_to = "spec") |>
  as_tibble() |>
  mutate(spec = vapply(spec, function(m) fabletools::model_sum(m$fit), character(1)))
   ↪  |>
  select(design, .model, spec)

ic <- glance(fit) |>
  filter(.model %in% arima_models) |>
  select(design, .model, sigma2, AICc)

order_tbl <- orders |> left_join(ic, by = c("design", ".model"))

order_tbl |>
  mutate(.model = model_labels[.model]) |>
  rename(Design = design, Model = .model, Specification = spec,
         `sigma^2 (log scale)` = sigma2) |>
  kable(digits = c(0, 0, 0, 4, 1))
get_spec  <- function(d, m) order_tbl$spec[order_tbl$design == d & order_tbl$.model
 ↪  == m]
get_sig2  <- function(d, m) order_tbl$sigma2[order_tbl$design == d & order_tbl$.model
 ↪  == m]
get_aicc  <- function(d, m) order_tbl$AICc[order_tbl$design == d & order_tbl$.model
 ↪  == m]
```

Table 5: Selected ARIMA orders, residual variance (log scale) and AICc for each design. AICc values are only comparable within a design, since the two designs are fitted to different data.

Design	Model	Specification	sigma^2 (log scale)	AICc
A: COVID kept	ARMA	ARIMA(1,0,1) w/ mean	0.1048	73.0
A: COVID kept	ARIMA	ARIMA(1,1,2)	0.1071	74.7
A: COVID kept	SARIMA (auto)	ARIMA(0,1,3)(2,1,0)[12]	0.1567	112.3

Table 5: Selected ARIMA orders, residual variance (log scale) and AICc for each design. AICc values are only comparable within a design, since the two designs are fitted to different data.

Design	Model	Specification	sigma^2 (log scale)	AICc
A: COVID kept	SARIMA (0,1,1)(0,1,1)	ARIMA(0,1,1)(0,1,1)[12]	0.1073	92.4
B: COVID as missing	ARMA	ARIMA(3,0,1) w/ mean	0.0038	-207.3
B: COVID as missing	ARIMA	ARIMA(0,1,4)	0.0039	-207.0
B: COVID as missing	SARIMA (auto)	ARIMA(0,1,1)(0,1,2)[12]	0.0007	-293.3
B: COVID as missing	SARIMA (0,1,1)(0,1,1)	ARIMA(0,1,1)(0,1,1)[12]	0.0008	-288.0

```
coef_tbl <- tidy(fit) |>
  filter(.model == "SARIMA_BJ") |>
  select(design, term, estimate, std.error)

kable(coef_tbl, digits = 3, col.names = c("Design", "Term", "Estimate", "Std.
 ↪  error"))
sma_A <- coef_tbl |> filter(design == design_levels[1], term == "sma1") |>
 ↪  pull(estimate)
sma_B <- coef_tbl |> filter(design == design_levels[2], term == "sma1") |>
 ↪  pull(estimate)
ma_A  <- coef_tbl |> filter(design == design_levels[1], term == "ma1")  |>
 ↪  pull(estimate)
ma_B  <- coef_tbl |> filter(design == design_levels[2], term == "ma1")  |>
 ↪  pull(estimate)
```

Table 6: Estimated coefficients of the hand identified SARIMA(0,1,1)(0,1,1) model under both designs.

Design	Term	Estimate	Std. error
A: COVID kept	ma1	0.280	0.110
A: COVID kept	sma1	-1.000	0.114
B: COVID as missing	ma1	-0.332	0.103
B: COVID as missing	sma1	-0.499	0.148

The two designs give very different models from the same data. Under Design A the residual variance of the airline model is 0.1073 on the log scale, against 0.0008 under Design B, about 134 times larger. The coefficients move too (Table 6). Under Design A the seasonal MA coefficient is -1.000. A value at or near -1 means the seasonal MA factor $(1 + \Theta_1 L^{12})$ almost cancels the seasonal difference $(1 - L^{12})$: the model is close to non-invertible and is effectively saying that the seasonal pattern is fixed and should not have been differenced. It also sits on the boundary of the parameter space, where the usual standard errors are unreliable. Under Design B the estimate is -0.499 and the non-seasonal MA coefficient is -0.332 (against 0.280 under Design A). The Design B values are inside the invertible region and agree in sign with the negative lag 1 and lag 12 autocorrelations found in the identification step.

Under Design B the automatic search chose ARIMA(0,1,1)(0,1,2)[12] with AICc -293.3, compared with -288.0 for the hand identified model. Whether the extra flexibility is worth it is answered by the test set below, not by the AICc.

5.2 Residual diagnostics

```
aug <- augment(fit) |> filter(.model %in% arima_models)

dof_tbl <- tidy(fit) |>
  filter(.model %in% arima_models, term != "constant") |>
  count(design, .model, name = "dof")

lb_tbl <- aug |>
  as_tibble() |>
  left_join(dof_tbl, by = c("design", ".model")) |>
  group_by(design, .model) |>
  summarise(
    dof     = first(dof),
    lb_stat = Box.test(.innov, lag = 24, type = "Ljung-Box", fitdf =
     ↪  first(dof))$statistic,
    lb_p    = Box.test(.innov, lag = 24, type = "Ljung-Box", fitdf =
     ↪  first(dof))$p.value,
    max_abs = max(abs(.innov), na.rm = TRUE) / sd(.innov, na.rm = TRUE),
    .groups = "drop"
  )

lb_tbl <- lb_tbl |>
  mutate(passes_lb = if_else(lb_p > 0.05, "Yes", "No"))

lb_tbl |>
  mutate(.model = model_labels[.model]) |>
  kable(digits = c(0, 0, 0, 2, 3, 1, 0),
        col.names = c("Design", "Model", "ARMA coefs", "Q*", "p-value",
                      "Max |residual| / sd", "Passes white noise test"))
```

Table 7: Ljung and Box tests on the innovation residuals (24 lags, degrees of freedom reduced by the number of ARMA coefficients), the largest absolute residual in standard deviation units, and whether the residuals pass the white noise test at the 5% level.

Design	Model	ARMA coefs	Q*	p-value	Max |residual| / sd	Passes white noise test
A: COVID kept	ARIMA	3	12.72	0.918	9.9	Yes
A: COVID kept	ARMA	2	12.54	0.945	9.8	Yes
A: COVID kept	SARIMA (0,1,1)(0,1,1)	2	13.69	0.912	8.9	Yes
A: COVID kept	SARIMA (auto)	5	13.92	0.788	8.3	Yes

Table 7: Ljung and Box tests on the innovation residuals (24 lags, degrees of freedom reduced by the number of ARMA coefficients), the largest absolute residual in standard deviation units, and whether the residuals pass the white noise test at the 5% level.

Design	Model	ARMA coefs	Q*	p-value	Max |residual| / sd	Passes white noise test
B: COVID as missing	ARIMA	4	117.73	0.000	2.9	No
B: COVID as missing	ARMA	4	113.93	0.000	2.7	No
B: COVID as missing	SARIMA (0,1,1)(0,1,1)	2	29.19	0.140	2.9	Yes
B: COVID as missing	SARIMA (auto)	3	32.63	0.051	2.8	Yes

```
aug |>
  filter(.model == "SARIMA_BJ") |>
  ggplot(aes(as.Date(Month), .innov)) +
  geom_hline(yintercept = 0, colour = "grey50") +
  geom_line(colour = "steelblue4") +
  facet_wrap(vars(design), ncol = 1, scales = "free_y") +
  labs(x = NULL, y = "Innovation residual (log scale)")
```

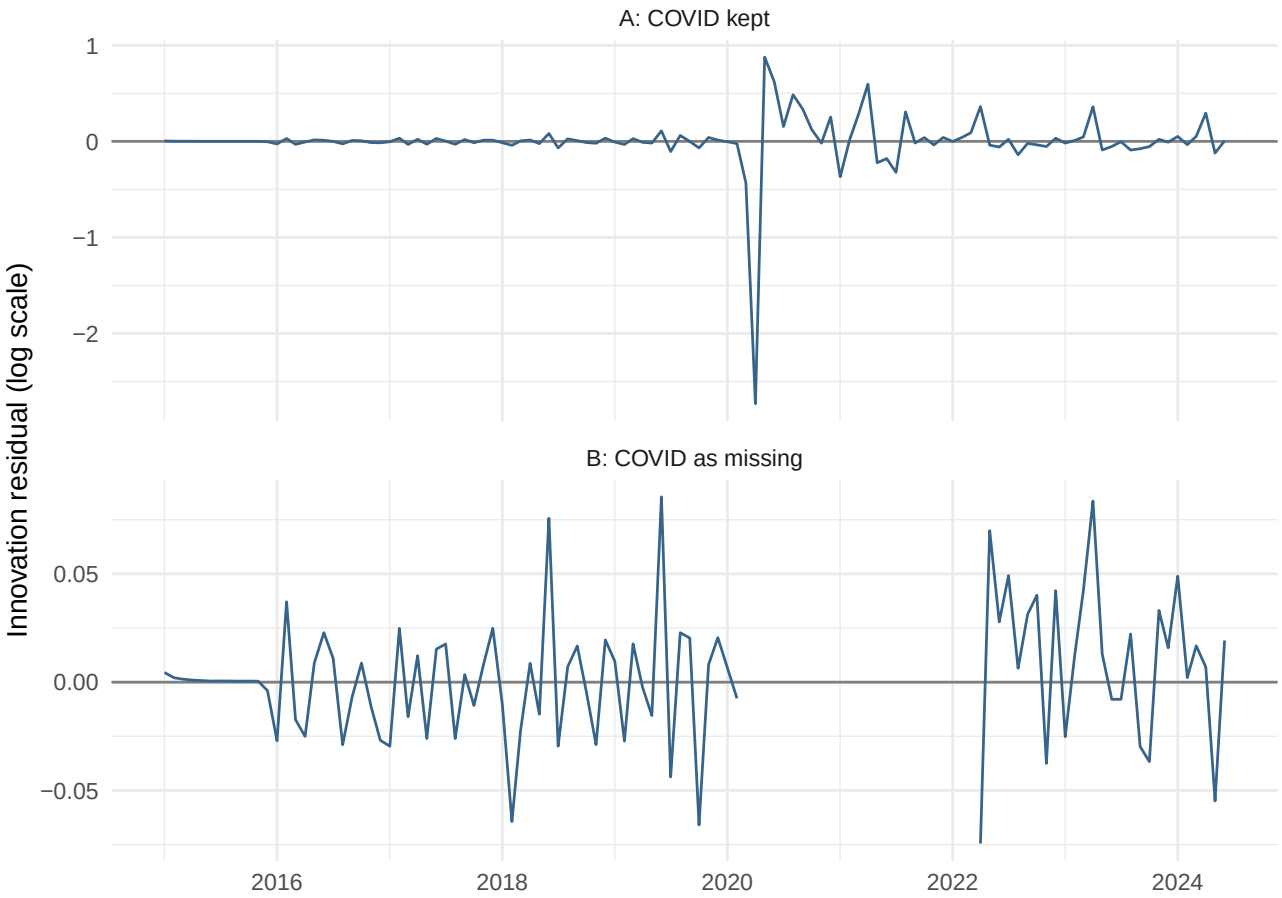


Figure 8: Innovation residuals of the SARIMA(0,1,1)(0,1,1) model under both designs. Under Design B the COVID window is missing, so there are no residuals for those months.

```
resid_B <- aug |>
  filter(.model == "SARIMA_BJ", design == design_levels[2])

p_acf <- resid_B |>
  ACF(.innov, lag_max = 36, na.action = na.pass) |>
  autoplot() + labs(title = "ACF")

p_hist <- resid_B |>
  ggplot(aes(.innov)) +
  geom_histogram(bins = 20, fill = "steelblue4", colour = "white") +
  labs(x = "Innovation residual", y = "Count", title = "Histogram")

if (requireNamespace("patchwork", quietly = TRUE)) {
  patchwork::wrap_plots(p_acf, p_hist, nrow = 1)
} else {
  print(p_acf); print(p_hist)
}
```

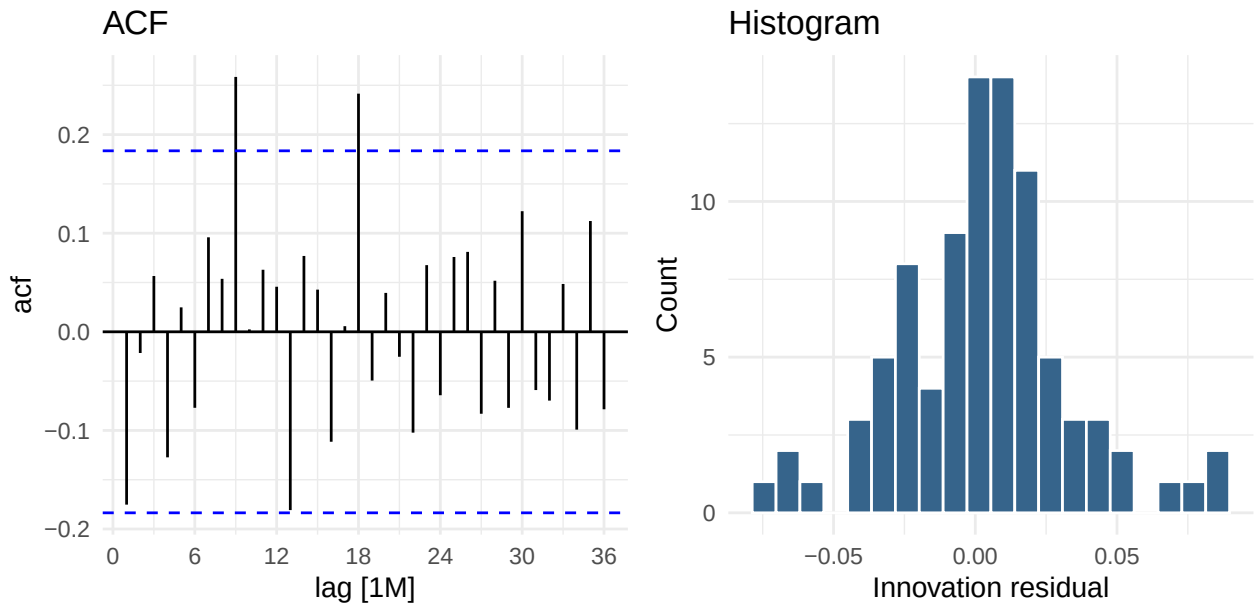


Figure 9: ACF and histogram of the Design B airline model residuals. The ACF is computed with missing values passed through, so the pre-COVID and post-COVID residuals are both used.

Under Design A the residual plot is dominated by a handful of enormous residuals in 2020 (the largest is 8.9 standard deviations from zero), so the normality assumption behind the prediction intervals fails badly, even though the Ljung and Box test ($p = 0.912$) does not detect autocorrelation. This is a useful reminder that a white noise test only checks the correlation structure: residuals can be uncorrelated and still be far from the identically distributed errors the model assumes. Under Design B the airline residuals look like white noise ($p = 0.140$), the ACF has no important spikes, and the histogram is roughly symmetric. The ARMA model without differencing leaves strong autocorrelation in its residuals under Design B ($p = 0.000$), which is expected: it has no way to represent the seasonal pattern.

Applying step 6 of the loop. Under Design B the models that pass the white noise test are: SARIMA (0,1,1)(0,1,1), SARIMA (auto). The models that fail are: ARIMA, ARMA. For a model that fails, the workflow says to return to step 3 rather than forecast. For the ARMA and non-seasonal ARIMA models, returning to step 3 means recognising from their residual ACF that the seasonal autocorrelation has not been modelled, which leads straight to the seasonal models. The automatically selected SARIMA passes as well, so it remains a candidate alongside the hand identified model. Under Design A every model technically passes the Ljung and Box test, but the airline model fails step 5 in a different way: its residuals are far from identically distributed (the COVID residuals are extreme outliers) and its seasonal MA coefficient sits on the invertibility boundary. Returning to step 3 under Design A would not fix this, because the problem is the data, not the orders. The fix is to change how the break is handled, which is what Design B does.

Candidates for the final model. Only the models that pass the loop under Design B are candidates for forecasting. The test set comparison below then checks the last condition from the notes: whether these candidates beat the benchmarks.

6 Forecast evaluation on the test set

```
fc <- fit |> forecast(h = "2 years")

acc <- fc |>
  accuracy(nights_keyed, measures = c(point_accuracy_measures,
   ↪  interval_accuracy_measures)) |>
  select(design, .model, RMSE, MAE, MAPE, MASE, winkler)

coverage <- fc |>
  hilo(level = c(80, 95)) |>
  unpack_hilo(c(`80%`, `95%`)) |>
  as_tibble() |>
  select(design, .model, Month, `80%_lower`, `80%_upper`, `95%_lower`, `95%_upper`)
   ↪  |>
  left_join(as_tibble(test) |> select(Month, actual = nights), by = "Month") |>
  group_by(design, .model) |>
  summarise(
    cover80 = mean(actual >= `80%_lower` & actual <= `80%_upper`),
    cover95 = mean(actual >= `95%_lower` & actual <= `95%_upper`),
    width95 = mean(`95%_upper` - `95%_lower`),
    .groups = "drop"
  )

results <- acc |>
  left_join(coverage, by = c("design", ".model")) |>
  arrange(design, RMSE)

results |>
  mutate(.model = model_labels[.model],
         cover80 = pct(cover80), cover95 = pct(cover95)) |>
  kable(digits = c(0, 0, 1, 1, 2, 2, 0, 0, 0, 0),
        col.names = c("Design", "Model", "RMSE", "MAE", "MAPE (%)", "MASE",
                      "Winkler (95%)", "Cover 80%", "Cover 95%", "Mean 95% width"),
        format.args = list(big.mark = ","))
```

Table 8: Test set accuracy, July 2024 to June 2026 (24 months). RMSE, MAE and the mean 95% interval width are in thousands of nights. MASE is scaled by the in-sample seasonal naive error of the observed training data, so it is comparable across designs. Coverage is the share of test months inside the 80% and 95% prediction intervals. Rows are sorted by RMSE within each design.

Design	Model	RMSE	MAE	MAPE (%)	MASE	Winkler (95%)	Cover 80%	Cover 95%	Mean 95% width
A: COVID kept	ARIMA	102.4	84.7	4.44	0.24	5,182	100%	100%	5,182
A: COVID kept	Mean	117.2	91.9	4.71	0.27	2,268	100%	100%	2,268

Table 8: Test set accuracy, July 2024 to June 2026 (24 months). RMSE, MAE and the mean 95% interval width are in thousands of nights. MASE is scaled by the in-sample seasonal naive error of the observed training data, so it is comparable across designs. Coverage is the share of test months inside the 80% and 95% prediction intervals. Rows are sorted by RMSE within each design.

Design	Model	RMSE	MAE	MAPE (%)	MASE	Winkler (95%)	Cover 80%	Cover 95%	Mean 95% width
A: COVID kept	ARMA	130.3	104.5	5.63	0.30	4,898	100%	100%	4,898
A: COVID kept	Seasonal naive	132.6	112.2	5.81	0.32	2,898	100%	100%	2,898
A: COVID kept	Naive	263.4	240.1	12.33	0.69	2,980	100%	100%	2,980
A: COVID kept	Drift	321.8	298.0	15.36	0.86	3,181	100%	100%	3,181
A: COVID kept	SARIMA (auto)	2,395.5	2,068.1	107.56	5.97	30,803	100%	100%	30,803
A: COVID kept	SARIMA (0,1,1)(0,1,1)	3,243.5	2,845.2	147.41	8.21	58,546	100%	100%	58,546
B: COVID as missing	SARIMA (0,1,1)(0,1,1)	101.6	81.4	4.25	0.23	545	92%	100%	545
B: COVID as missing	ARMA	121.9	99.8	5.17	0.29	872	100%	100%	872
B: COVID as missing	Seasonal naive	132.6	112.2	5.81	0.32	1,049	100%	100%	1,049
B: COVID as missing	SARIMA (auto)	152.9	129.4	6.74	0.37	595	88%	96%	542
B: COVID as missing	ARIMA	196.3	166.7	8.48	0.48	701	71%	100%	701
B: COVID as missing	Mean	229.6	205.2	11.12	0.59	1,140	92%	100%	1,140
B: COVID as missing	Naive	263.4	240.1	12.33	0.69	2,314	100%	100%	2,314
B: COVID as missing	Drift	338.9	314.4	16.22	0.91	2,512	100%	100%	2,512

```
fc |>
  filter(.model %in% c("SNaive", "SARIMA_auto", "SARIMA_BJ")) |>
  autoplot(nights_keyed |> filter_index("2021 Jan" ~ .), level = 95) +
  facet_wrap(vars(design), ncol = 1, scales = "free_y") +
  labs(x = NULL, y = "Nights sold (thousands)", colour = "Model", fill = "Model")
```

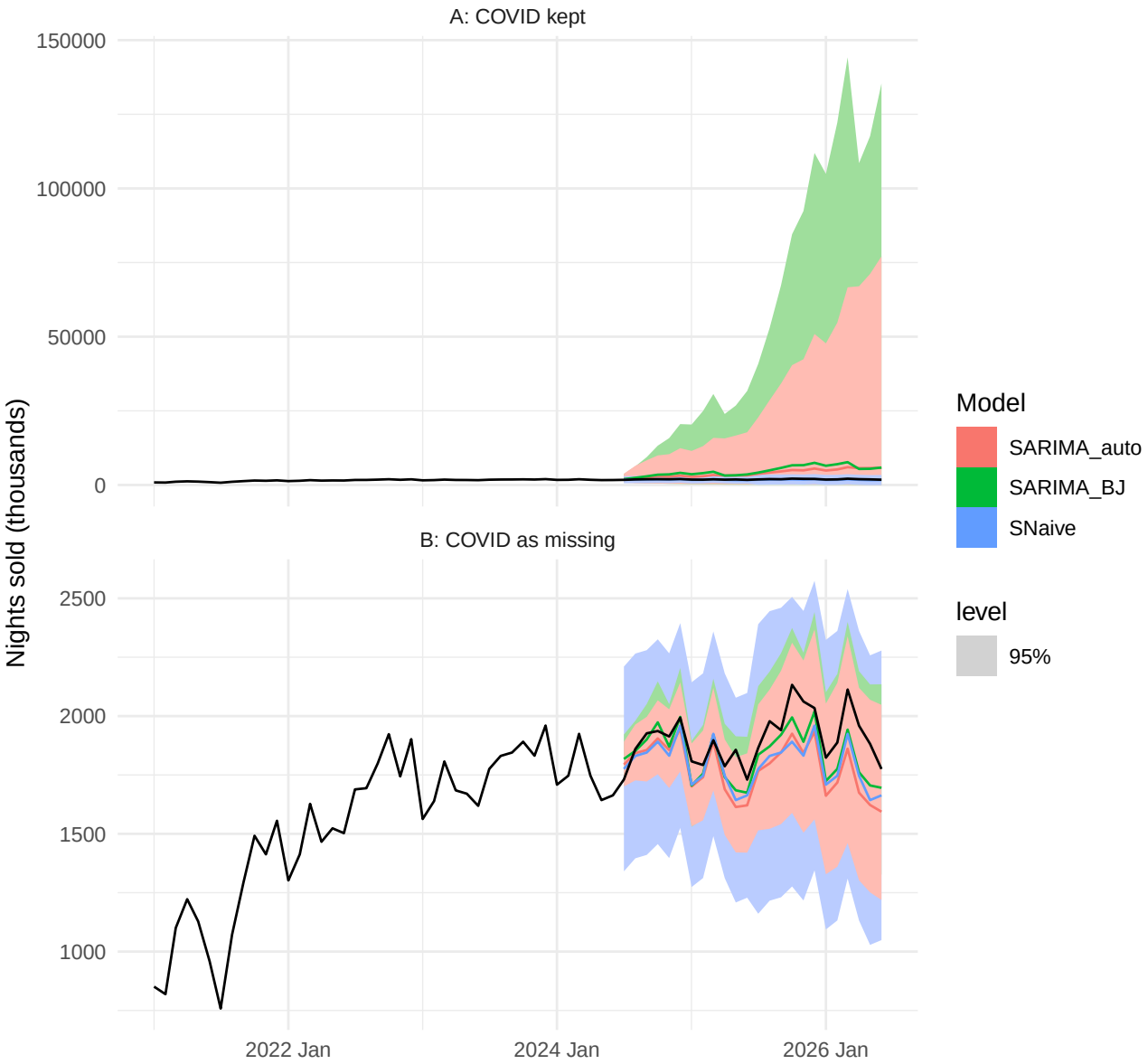


Figure 10: Test set forecasts with 95% prediction intervals from the seasonal naive benchmark (SNaive), the automatic SARIMA (SARIMA_auto) and the airline model (SARIMA_BJ), under both designs. Black lines are the observed values. Under Design A the intervals are so wide that they dominate the vertical axis.

Design A. With the COVID months kept, the most accurate model on the test set is the ARIMA model. The airline SARIMA has an RMSE of 3,243.5 thousand nights against 132.6 for the seasonal naive method, and the automatically selected ARIMA(0,1,3)(2,1,0)[12] has an RMSE of 2,395.5. In other words, the more sophisticated models do not earn their extra parameters. Part of the reason is visible in Figure 10: the residual variance inflated by 2020 makes the 95% intervals extremely wide (mean width 58,546 thousand nights for the airline model), and because the bias adjustment adds a term proportional to $\hat{\sigma}^2$ when back-transforming from the log scale, the inflated variance also pushes the point forecasts upwards. The intervals reach 100% coverage, but only because they are uninformatively wide; the large Winkler scores penalise this.

Design B. With the COVID window treated as missing, the ranking changes. The most accurate model is the SARIMA (0,1,1)(0,1,1) model. The airline SARIMA reaches an RMSE of 101.6 and a MAPE of 4.25%, compared with 132.6 for the best benchmark (Seasonal naive) and 132.6 for seasonal naive. Its 95% intervals have a mean width of 545 thousand nights, much narrower than under Design A, and still cover 100% of the test months (92% for the 80% intervals). The automatically selected model has lower AICc than the airline model but higher test RMSE (152.9), which shows that an in-sample criterion and out-of-sample accuracy do not have to agree.

The model ladder. Under Design B, moving from ARMA (RMSE 121.9) to ARIMA (196.3) does not help on its own: differencing removes the level but a non-seasonal model still cannot reproduce the December peak, so its forecasts flatten out after a few months. The large gain comes only when the seasonal structure is modelled explicitly with the SARIMA. The MASE values below 1 or above 1 should be read with care: the scaling uses the in-sample seasonal naive error over the whole training period, which includes the COVID collapse, so all MASE values are small in absolute terms; they are useful for ranking, not as an absolute measure.

7 Rolling-origin cross-validation

A single test set is only one forecast origin, and the ranking above could change if the split moved by a few months. Following the notes, the comparison is repeated with an expanding window: the first origin is June 2023 and the origin moves forward one month at a time until June 2025, giving 25 origins. At each origin the models are refitted on all data up to that month and used to forecast 1 to 12 months ahead. The orders of the SARIMA models are kept fixed at each refit so that the exercise measures forecasting performance rather than the order selection procedure.

```
cv_data <- bind_rows(
  as_tibble(nights_sold) |> mutate(design = design_levels[1]),
  as_tibble(nights_sold) |>
    mutate(design = design_levels[2],
           nights = if_else(period == "COVID window", NA_real_, nights))
) |>
  as_tsibble(index = Month, key = design)

init_n <- nights_sold |> filter(Month <= yearmonth("2023 Jun")) |> nrow()

cv_stretch <- cv_data |>
  filter(Month <= yearmonth("2025 Jun")) |>
  stretch_tsibble(.init = init_n, .step = 1)

cv_fit <- cv_stretch |>
  model(
    Mean      = MEAN(nights),
    SNaive    = SNAIVE(nights),
    SARIMA_BJ = ARIMA(log(nights) ~ 0 + pdq(0, 1, 1) + PDQ(0, 1, 1))
  )

cv_fc <- cv_fit |>
  forecast(h = 12) |>
  group_by(design, .model, .id) |>
```

```
  mutate(h = row_number()) |>
  ungroup() |>
  as_fable(response = "nights", distribution = nights)

cv_acc_h <- cv_fc |>
  accuracy(nights_keyed, by = c("h", ".model", "design"), measures = list(RMSE =
    ↪  RMSE))

cv_acc_all <- cv_acc_h |>
  group_by(design, .model) |>
  summarise(`Mean RMSE over h = 1 to 12` = mean(RMSE), .groups = "drop")

cv_acc_all |>
  mutate(.model = model_labels[.model]) |>
  arrange(design, `Mean RMSE over h = 1 to 12`) |>
  kable(digits = 1, col.names = c("Design", "Model", "Mean RMSE (h = 1 to 12)"))
```

Table 9: Rolling-origin cross-validation: RMSE averaged over horizons 1 to 12 (thousands of nights).

Design	Model	Mean RMSE (h = 1 to 12)
A: COVID kept	Seasonal naive	96.2
A: COVID kept	Mean	118.2
A: COVID kept	SARIMA (0,1,1)(0,1,1)	1277.9
B: COVID as missing	SARIMA (0,1,1)(0,1,1)	76.0
B: COVID as missing	Seasonal naive	96.2
B: COVID as missing	Mean	280.0

```
cv_acc_h |>
  mutate(.model = model_labels[.model]) |>
  ggplot(aes(h, RMSE, colour = .model)) +
  geom_line() +
  geom_point(size = 1.2) +
  facet_wrap(vars(design), nrow = 1) +
  scale_x_continuous(breaks = 1:12) +
  scale_y_log10(labels = scales::label_comma()) +
  labs(x = "Forecast horizon (months)", y = "RMSE (log scale)", colour = "Model")
```

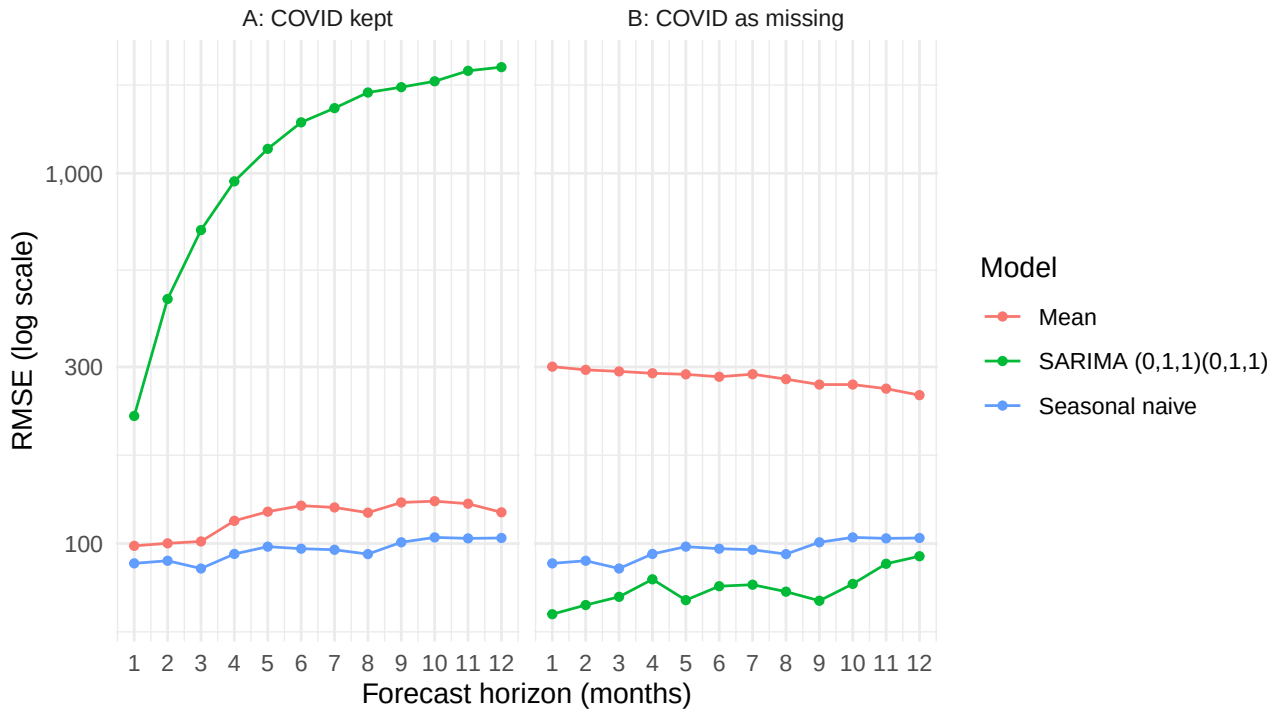


Figure 11: Cross-validated RMSE by forecast horizon. A log scale is used because the Design A SARIMA errors are an order of magnitude larger than the others.

Cross-validation confirms the test set conclusions and makes the contrast between the designs sharper. Under Design A the airline model averages an RMSE of 1,277.9 across horizons, far worse than seasonal naive (96.2), and its error grows quickly with the horizon (Figure 11). The reason is the bias adjustment again: with $\hat{\sigma}^2$ inflated by the COVID months, the variance of a log scale forecast grows with h, and so does the upward adjustment to the back-transformed mean. Under Design B the same model has the lowest error at every horizon, with an average RMSE of 76.0 against 96.2 for seasonal naive and 280.0 for the mean method. Neighbouring origins share most of their data, so these 25 origins are not independent replicates, but the ranking is consistent across every horizon.

8 Final model and forecasts

The final model is the SARIMA$(0,1,1)(0,1,1)_{12}$ on log nights sold with the COVID window treated as missing, refitted to all data up to June 2026.

```
final_data <- nights_sold |>
  mutate(nights = if_else(period == "COVID window", NA_real_, nights))

final_fit <- final_data |>
  model(SARIMA_BJ = ARIMA(log(nights) ~ 0 + pdq(0, 1, 1) + PDQ(0, 1, 1)))

report(final_fit)
```

```
Series: nights
```

```
Model: ARIMA(0,1,1)(0,1,1)[12]
Transformation: log(nights)

Coefficients:
          ma1     sma1
      -0.4105  -0.6205
s.e.   0.0915   0.1417

sigma^2 estimated as 0.0009069:  log likelihood=191.71
AIC=-377.42   AICc=-377.22   BIC=-368.93
```

```
final_fc <- final_fit |> forecast(h = "2 years")
```

```
final_fc |>
  autoplot(nights_sold |> filter_index("2022 Apr" ~ .), level = c(80, 95)) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = NULL, y = "Nights sold (thousands)")
```

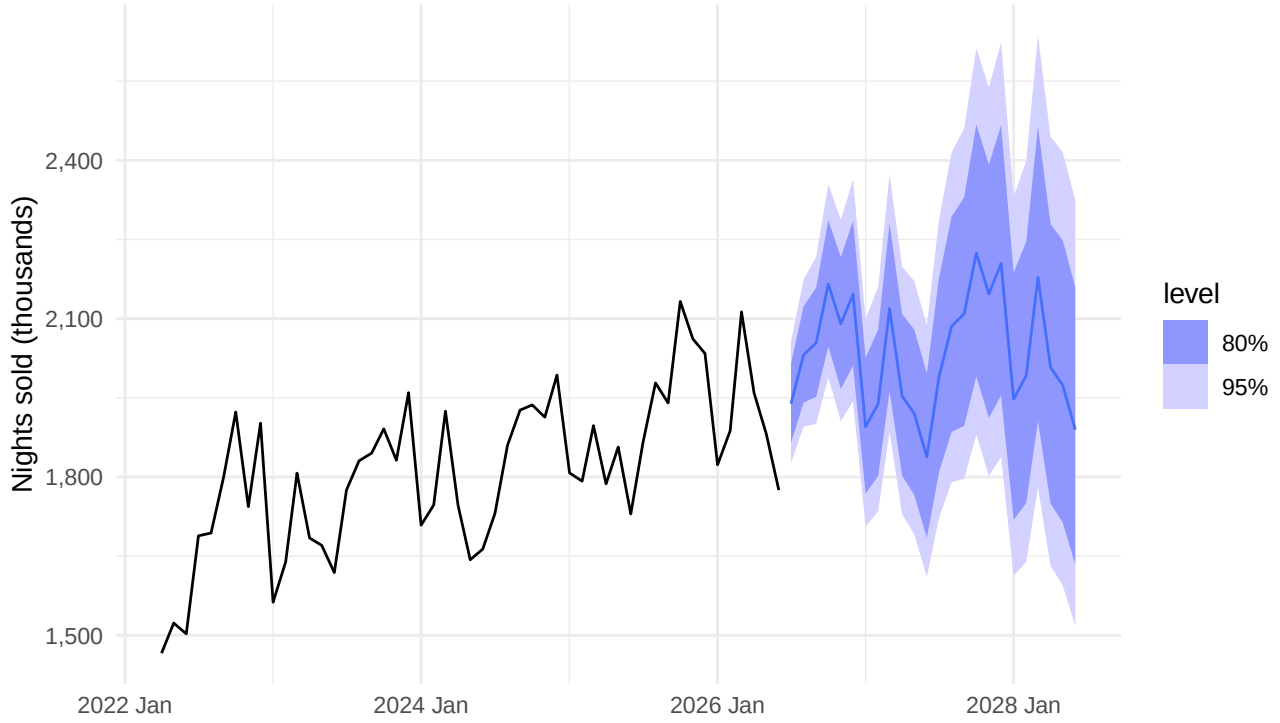


Figure 12: Forecasts of monthly stay unit nights sold, July 2026 to June 2028, from the final model, with 80% and 95% prediction intervals.

```
final_fc |>
  hilo(level = 95) |>
  unpack_hilo(`95%`) |>
  as_tibble() |>
  filter(month(as.Date(Month)) %in% c(1, 4, 7, 10, 12)) |>
  transmute(Month = format(Month), Forecast = .mean,
            `Lower 95%` = `95%_lower`, `Upper 95%` = `95%_upper`) |>
  kable(digits = 0, format.args = list(big.mark = ","))
```

Table 10: Selected point forecasts and 95% prediction intervals (thousands of nights).

Month	Forecast	Lower 95%	Upper 95%
2026 Jul	1,939	1,827	2,056
2026 Oct	2,166	1,989	2,355
2026 Dec	2,146	1,944	2,364
2027 Jan	1,895	1,706	2,100
2027 Apr	1,953	1,729	2,198
2027 Jul	1,990	1,723	2,287
2027 Oct	2,224	1,880	2,612
2027 Dec	2,205	1,838	2,623
2028 Jan	1,947	1,613	2,331
2028 Apr	2,008	1,632	2,444

The model projects the familiar seasonal shape forward on a level close to that of the last two years, with December 2026 Dec forecast at 2,146 thousand nights (95% interval 1,944 to 2,364). The intervals widen with the horizon because the model contains two unit roots, so uncertainty about the level accumulates. These intervals assume that no new shock of the COVID kind occurs; they describe ordinary month-to-month uncertainty only.

9 Discussion

9.1 Key findings

1. **Seasonality is the dominant and most stable feature of the series.** The seasonal shape barely changed even through the COVID years, which is why the seasonal naive method is hard to beat and why the non-seasonal ARMA and ARIMA models perform poorly.
2. **How the break is handled matters more than which ARIMA model is chosen.** With the COVID months kept (Design A), the seasonal ARIMA models were worse than the seasonal naive benchmark on the test set and much worse in cross-validation. The cause is a residual variance inflated by about 134 times, a seasonal MA coefficient pushed to the invertibility boundary (-1.000), and point forecasts pushed upwards by the bias adjustment.
3. **With the COVID window treated as missing (Design B), the Box and Jenkins airline model was the only model to pass the full loop and beat the benchmarks**: it passed the diagnostic step and performed best, with a test RMSE of 101.6 thousand nights (MAPE 4.25%), white noise residuals and well calibrated prediction intervals. It also had the lowest cross-validated error at every horizon from 1 to 12 months.
4. **A lower AICc did not guarantee better forecasts.** The automatically selected seasonal model had a lower AICc than the hand identified model but higher test RMSE, so the simpler model identified from the ACF and PACF was preferred.

9.2 Issues and limitations

- **Choice of COVID window.** The window (March 2020 to March 2022) follows the national state of disaster. The data suggest the recovery continued after April 2022, so some post-COVID

months still carry the recovery trend into the training data. A sensitivity check with a window ending in December 2022 would test how much this matters.

- **Treating the shock as missing discards information.** Setting 25 months to missing throws away real data, and the approach would not work for a shock at the very end of a series. An intervention model (a level shift or a ramp with a gradual recovery) would use those months instead of ignoring them; this is the starting point for Project 3.
- **Level shift after COVID.** Demand has recovered to slightly below its pre-COVID level. The differenced models adapt to this because they forecast from the most recent level, but the mean benchmark only did reasonably well because the recovered level happens to sit near the long-run average, which would not hold in general.
- **MASE scaling.** The MASE denominator uses an in-sample period that contains the COVID collapse, so the MASE values are deflated. RMSE and MAPE were used as the main measures.
- **Bias adjustment.** Back-transforming with the mean (bias adjusted) forecast is the right choice when the forecasts are added up, for example into annual totals, but it makes the forecasts very sensitive to an inflated $\hat{\sigma}^2$. This was a large part of the failure of Design A and is easy to miss.
- **Short post-COVID history.** Only about two years of post-COVID data are in the training set, and the test set is two years long, so the conclusions rest on a fairly short stretch of "normal" data.

9.3 What I would do differently

I would model the COVID shock directly with intervention variables in a regression with ARIMA errors, so that the 25 months are used rather than discarded, and compare a sliding window with the expanding window, since a sliding window is the natural choice when structural breaks are plausible. Project 3 takes up both ideas on the same series and the same test set, which allows a direct comparison with the SARIMA results from this project.

Reproducibility

All results in this document are produced by the code above from the Stats SA P6410 spreadsheet (`tourist_data.xlsx`). The random seed is fixed, although none of the methods used here are random.

```
R version 4.5.3 (2026-03-11 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_South Africa.utf8  LC_CTYPE=English_South Africa.utf8
[3] LC_MONETARY=English_South Africa.utf8 LC_NUMERIC=C
[5] LC_TIME=English_South Africa.utf8

time zone: Africa/Johannesburg
tzcode source: internal

attached base packages:
```

```
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] knitr_1.51        readxl_1.5.0      fable_0.5.0       feasts_0.5.0
 [5] fabletools_0.8.0  ggtime_0.2.0      tsibbledata_0.4.1 tsibble_1.2.0
 [9] ggplot2_4.0.3     lubridate_1.9.5   tidyr_1.3.2       dplyr_1.2.0
[13] tibble_3.3.1      fpp3_1.0.3

loaded via a namespace (and not attached):
 [1] ggdist_3.3.3         rappdirs_0.3.4       generics_0.1.4
 [4] anytime_0.3.13       lattice_0.22-9       digest_0.6.39
 [7] magrittr_2.0.4       evaluate_1.0.5       grid_4.5.3
[10] timechange_0.4.0     RColorBrewer_1.1-3   fastmap_1.2.0
[13] cellranger_1.1.0     jsonlite_2.0.0       purrr_1.2.2
[16] scales_1.4.0         numDeriv_2016.8-1.1  cli_3.6.5
[19] rlang_1.1.7          crayon_1.5.3         withr_3.0.3
[22] yaml_2.3.12          otel_0.2.0           tools_4.5.3
[25] vctrs_0.7.1          R6_2.6.1             lifecycle_1.0.5
[28] pkgconfig_2.0.3      urca_1.3-4           progressr_1.0.0
[31] pillar_1.11.1        gtable_0.3.6         glue_1.8.0
[34] Rcpp_1.1.2           xfun_0.56            tidyselect_1.2.1
[37] rstudioapi_0.19.0    farver_2.1.2         patchwork_1.3.2
[40] htmltools_0.5.9      nlme_3.1-168         rmarkdown_2.31
[43] labeling_0.4.3       compiler_4.5.3       S7_0.2.1
[46] distributional_0.8.1
```